

Towards Decentralized Sphinx Header Construction

Aurélien Chassagne
Université Libre de Bruxelles
Brussels, Belgium
aurelien.chassagne@ulb.be

Jan Tobias Mühlberg
Université Libre de Bruxelles
Brussels, Belgium
jan.tobias.muehlberg@ulb.be

Iness Ben Guirat
Université Libre de Bruxelles
Brussels, Belgium
iness.ben.guirat@ulb.be

Jean-Michel Dricot
Université Libre de Bruxelles
Brussels, Belgium
jean-michel.dricot@ulb.be

Abstract

To update

We propose a decentralized scheme that prevent mixnets users from sending traffic that does not match the service provider of an anonymous credential. Our scheme is of direct use in the Nym network where users construct an anonymous credential given they receive a certificate of them paying for that service provider. Our scheme prevent users from cheating by using a credential for a free service and send traffic for a paid service. Our solution works even if the majority of the third parties collude. Finally we evaluate the performance of our solution.

CCS Concepts

• **Security and privacy** → *Privacy-preserving protocols; Distributed systems security.*

Keywords

mixnet, sphinx, malicious users

ACM Reference Format:

Aurélien Chassagne, Iness Ben Guirat, Jan Tobias Mühlberg, and Jean-Michel Dricot. 2025. Towards Decentralized Sphinx Header Construction. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (WPES '25)*. ACM, New York, NY, USA, 9 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

Mix network, or mixnet, is an overlay network of servers, called mixnodes, that prevent an adversary from correlating senders with receivers [3, 4, 6, 8, 11, 14, 16, 17]. They achieve this by adding delays to messages inside the different mixnodes in order to mitigate timing analysis attacks. Additionally, unlike Tor [10] where all packets sent by a user follow the same path for the entire session (circuit-based), in mixnets each packet follows a different routing routing path (packet-based). These two techniques ensure that mixnets are

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WPES '25, Taipei, Taiwan

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXX.XXXXXXX>

resilient against a strong adversary who observes the entire inputs and outputs of the network, typically called a Global Passive Adversary (GPA). During the last few decades several systems have been proposed in the literature and few of them have been implemented. Open research problems such as parameterization, authentication and fake dummies packets have halted the usage on a large scale.

I guess from JT: Also this speculation, needs rephrasing + citations
Aure: Iness I let you handle this because you are more competent than me on the subject and I am not sure to know what you are referring to...

Recently, Nym Technologies a mixnet-based company is being commercialized and proposing a mixnet network for services to be integrated with their network for a fee. Their network is based on Loopix [17]. Clients of these services are then allowed to use the Nym mixnet by using Nym credentials (based on the Coconut credential [18]) that they can construct after getting a certificate of paying for a specific service to use the Nym network. However, nothing prevents clients to exploit a valid credential to access services for which they have not paid for. Such misuse would only be detected at the mixnet's exit gateway, resulting in wasted resources for the intermediate nodes. This is a particular difficult problem in anonymous communication networks where the mixnodes do not know the traffic type or the final service a client is communicating with by doing layered encryption (Sphinx packet [7]) where the final IP address is only known at the exit gateway.

In this paper we present a scheme that creates the Sphinx header in a decentralized way based on trusted third parties while ensuring that these parties learn nothing about the destination or the path. Having a decentralized scheme offers greater flexibility for real-world deployment and facilitates ongoing improvements of the technology and its features. We think a decentralized approach could simplify the creation of credentials and zero-knowledge proofs that demonstrate the client has actually used an authorized destination as the final destination (which is encrypted under multiple layers). Our work makes the following contributions:

- We propose a decentralized approach to Sphinx header construction;
- We evaluate the unlinkability property of both the original Sphinx packet and our decentralized construction by assessing their randomness using the NIST SP 800-22 statistical test suite.

We highlight related work and motivation in Section 2, then we specify and justify our system model in Section 3, where we also describe the considered threat model. We then present our scheme

that decentralize the creation of the Sphinx headers in Section 4 and the evaluation of our proposed solution in Section 5. Finally we conclude and discuss future work in Section 6

2 Motivation and Related Work

Since Chaum's seminal work on untraceable email in 1981 [3], there has been a great amount of research related to mixnets' design [1, 3–6, 11–13, 17]. However most systems that have been deployed and used come with their own system on top of the network meaning that only one type of traffic is allowed. As stated by Dingledine et al. in [9], "Anonymity Loves Company" meaning that the more messages there are in the network the more privacy the network provide. This idea is also supported by Ben Guirat et al. [2], who show that blending different type of traffic over a mixnet provide better anonymity. For example, consider an instant messaging system where users expect latencies of only a few seconds, and an email application where users tolerate delays of up to a minute. The authors show that mixing these two types of traffic can actually improve the privacy of both traffic. This strategy is feasible only in networks like Tor or mixnet, which are designed to support multiple applications, rather than being tied to a single type of traffic.

However, several open research problems remain. For example, how can we prevent certain types of traffic, for whatever reason, without compromising privacy guarantees?

Should we be more specific and gives some concrete reasons?

Tor addresses this issue through exit policies, which allow exit nodes to simply drop unwanted traffic. This works in Tor because it is circuit-based meaning that all traffic from a specific user session follow one path and hence packets are encrypted using symmetric cryptography which is cheap. As a result, dropping traffic at the exit node does not waste significant computation power.

In contrast, mixnets present a more complex challenge. Due to packet-based routing, where each packet takes a different path, mixnets rely on layered public-key cryptography, which is computationally intensive. Dropping packets at the final node, which is typically the only point where service policies can be applied, results in a significant waste of computational resources spent by intermediate nodes on processing and forwarding those packets. Moreover, unlike Tor where relays operate on a voluntary basis, mixnets such as Nym use economic incentives through payment per relayed bandwidth. Consequently, dropping packets at the exit node not only wastes computational resources but also impacts the economic sustainability of the network.

This work raises several concrete research questions:

- Could the construction of Sphinx headers be effectively decentralized without compromising core security properties such as unlinkability and integrity?
- What are the computational and communication overhead introduced by the decentralization?
- Do those overhead costs are preferable to the bandwidth and computational waste caused by malicious users?
- Does decentralization facilitate the integration of advanced cryptographic mechanisms, such as zero-knowledge proofs, for credential verification?

3 System and Threat Model

I am not convinced of this section (especially this paragraph and the 'System Model' subsection). Moreover I feel like the mixnet figure and explanation is maybe not appropriate (or at the right place). What do you think of it?

Nym uses fixed-size Sphinx packets, which ensure that external observers cannot correlate incoming and outgoing messages at mixnodes. For example, if Signal is integrated with Nym, clients who want their traffic to be anonymous would route their traffic through the mixnet instead of connecting directly to the Signal server. This prevents adversaries monitoring either the client's device, the server, or any intermediate nodes from correlating sender and recipient. In addition, each Sphinx encryption layer includes an integrity tag. This ensures that any tampering with the packet is detectable, allowing mixnodes to drop corrupted messages early.

Each Nym client receives anonymous credentials (e.g. Coconut credentials [18]) to use the mixnet to route their traffic towards specific destinations (e.g. services) they have paid for. A potential vulnerability arises when a client obtains a credential for a specific service (e.g. Signal) but uses it to inject traffic directed to an unauthorized or unrelated service. Because the true destination is only revealed at the final hop of the mixnet, earlier mixnodes waste computational resources processing a packet that will ultimately be dropped. This misuse opens the door to denial-of-service (DoS) attacks, where malicious clients flood the system with unauthorized packets that will be detected at the last hop, draining bandwidth and compute capacity.

To address this, we propose a decentralized construction of Sphinx headers involving a set of trusted third parties (TTPs). This decentralized could enable TTPs to update credentials and/or zero-knowledge proofs in accordance with the layered encrypted header, to be verifiable any mixnodes on the route. As a result, mixnodes could early discard unauthorized traffic before it consumes significant resources.

In this section, we present our system model and threat model, and outline the privacy and security requirements our solution should satisfy.

3.1 System Model

As illustrated in Figure 2, our model involves three types of entities: the Client, a set of Trusted Third Parties (TTPs), and the Mixnet. The client is considered as potentially malicious. TTPs are assumed to be honest-but-curious, meaning that they correctly follow the protocol but may attempt to collude in order to extract additional information from the data they process. Mixnodes are responsible for enforcing access policies by dropping unauthorized traffic.

We assume that clients already possess valid Nym credentials that authorize them to access specific services. To send a message to such a service, the client first randomly selects a route through the mixnet. Next, the client computes a set of cryptographic elements, distributes and shares them among a set of TTPs. Each TTP then uses its shares to compute a partial Sphinx header and update the credential accordingly. The client aggregates these partial headers and credentials into a complete Sphinx header along with a valid credential. This final packet is then ready to be transmitted through

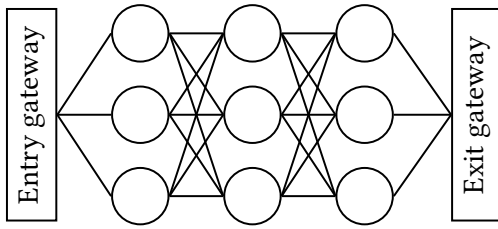


Figure 1: Mixnet architecture.

the mixnet. Note that credential issuance, modification, and verification are outside the scope of this paper and are left for future work.

A simplified view of a mixnet architecture is presented in Figure 1. A mixnet is a set of nodes organized in layers. Thereby each packet traverses a fixed and predetermined number of mixnodes. A packet enters through an entry node, is forwarded to a next layer's node and so on until it exits via an exit node. Each mixnode decrypts, reorders and delays packets before forwarding them to the next layer, breaking any direct correlation between input and output.

3.2 Threat Model

We consider different types of adversaries:

- **Global Passive Adversary (GPA):** An adversary able to observe all traffic entering and leaving the network and each mixnode. The GPA aims to correlate incoming and outgoing packets. Our construction ensures *bitwise unlinkability* of Sphinx packets, such that packet headers appear uniformly random and indistinguishable from one another, thereby preventing correlation.
- **Global Active Adversary (GAA):** An adversary able of modifying or injecting packets across the entire network. The GAA attempts to alter Sphinx headers in transit or to create a valid Sphinx header. Thanks to the integrity-check mechanisms embedded in the packet format, any such tampering is detected and causes the packet to be discarded by mixnodes.

- **Colluding Header Constructors:** In our decentralized header construction schema, multiple trusted third parties (TTPs) participate in generating the Sphinx headers. We assume that a subset of these parties may collude. Our design guarantees that even if a majority of these TTP collude, they learn nothing about the final destination or the route, preserving route confidentiality.
- **Malicious Clients:** We consider clients attempting to build Sphinx headers that misrepresent their intended destination by exploiting a valid credential to access a service for which they are not authorized. Such misuse can typically only be detected at the mixnet's exit gateway, where the true destination is revealed and can be compared against the credential. Our decentralized scheme could enable the

adaptation of those credentials such that they could be verified at any step in the network.

3.3 Security & Privacy Objectives

Our main security and privacy objectives are the following:

- **Unlinkability:** Our scheme aims to ensure that a global adversary cannot link incoming packets to their corresponding outgoing packets at any mixnode. The original Sphinx packet format introduced by Danezis and Goldberg [7] is known to achieve strong unlinkability guarantees. We target at least the same level of unlinkability, but adopt a relaxed version called *bitwise unlinkability*. If packets appear uniformly random and indistinguishable from one another, we consider that this prevents correlations between incoming and outgoing packets, thereby satisfying the unlinkability requirement.
- **Integrity:** Packets should be protected against tampering. Our schema should adapt or provide a similar integrity-check mechanisms as the original Sphinx, ensuring that any modification in the header is detectable by mixnodes.
- **Collusion Resistance:** Even if a majority of the Trusted Third Parties (TTPs) involved in header construction collude, they must not learn any information about the message, route or cryptographic secrets. Leakage of routing information could enable timing or metadata correlation attacks across different hops. Therefore, collusion resistance is necessary to maintain strong anonymity and metadata protection in our decentralized scheme.
- **Verifiability:** In the original Sphinx design, credentials can only be validated at the exit gateway, once the true destination is revealed. However, credentials should be verifiable at any step in the network, allowing for early detection and discarding of unauthorized packets, reducing wasted computational and bandwidth resources.

4 Our Solution - Multi-Party Computation (MPC)

Our approach to ensure trust in the Sphinx header is to prevent user manipulation by decentralizing the header construction to Trusted Third Parties (TTPs) through the use of Multi-Party Computation (MPC).

To facilitate explanation and illustration, we describe our solution such that each packet goes through a fixed-length path consisting of three mixnodes. However, the scheme is general and can be adapted to support arbitrary path lengths.

Better in 'model' section or here (since it's an assumption to have lighter math formula)

4.1 Header structure

In our approach, each piece of header information is encoded as an elliptic curve point. To achieve this, the encoded string is divided into fixed-size chunks, as illustrated in Figure 3, where each chunk is a single EC point. For example, in the case of a path consisting

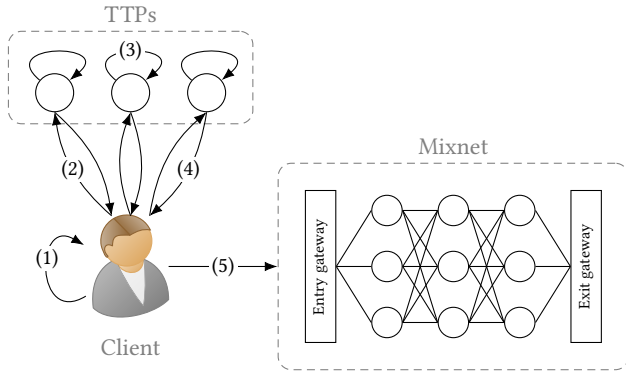


Figure 2: Overview of the decentralized scheme

of three mixnodes, the resulting header contains seven EC points: one destination, three mixnodes and three integrity tags.

Therefore a method to encode and decode information to and from elliptic curve points is required. Specifically, we require a mapping from integers to curve points, noted $\text{Map}()$, and the inverse mapping from curve points to integers, noted $\text{Map}^{-1}()$, such that $\text{Map}^{-1}(\text{Map}(x)) = x$ where x is an integer.

Traditional methods achieve this by directly mapping integers to the x-coordinates of points on the curve. However, this approach could leak information by introducing bias since nearby integers result in nearby points.

To address this, we adopt Elligator, which offers stronger privacy guarantees. Unlike the traditional approach, Elligator produces a uniformly distributed output which is computationally indistinguishable from truly random curve points. This uniformity is in line with our objectives of preserving anonymity and avoiding linkability.

4.2 Protocol description

The overall decentralized scheme is illustrated in Figure 2. (1) The client first computes a sequence of shared secrets and then splits these secrets, along with the IP addresses of the mixnodes in the path and the final destination, into m shares. (2) Each set of shares is sent to a different TTP, along with the necessary cryptographic element (α). (3) Each TTP independently computes a *partial* Sphinx header using the received shares. (4) The client then aggregates these partial headers to reconstruct the final header, (5) ready for transmission through the mixnet.

In summary, the protocol is divided into four main steps:

- **Setup:** fixes random points as generators (once but should be refreshed);
- **Client:** splits and shares the necessary information to TTPs;
- **TTP:** encrypts the routing information into a partial header;
- **Mixnode:** decrypts and forwards the header.

4.2.1 Setup. In our protocol, routing information is divided into seven chunks, each one encoded into an elliptic curve point (see Figure 3). These points will later be encrypted by adding a masking point of the form $P = sG$, where s is a shared secret and G is the base generator of the curve. However, using the same masking point P

for all the seven chunks compromises the *unlinkability* property (see security note in 4.2.3).

To mitigate this, we use a set of seven *independent* generators G_j , one for each chunk. By *independent*, we mean that the scalar relationship between any two generators is unknown. Therefore no one knows the scalars $z_j \in \mathbb{Z}_N$ such that $G_j = z_j G$.

In principle, this setup phase can be executed only once. However, to preserve unlinkability, the set of generators should be refreshed periodically. Every entity (clients, mixnodes, and TTPs) must run the setup phase to ensure they use the same set of generators.

To achieve synchronized and deterministic generators across the system, we derive them from a common seed. This seed is computed as the hash of the current timestamp, truncated to the chosen refresh interval. To produce *independent* generators, we simply increment the seed since Elligator already provides a uniform (i.e. pseudo-random) integers to points mapping.

A subtlety with Elligator is that it can map integers to points outside of the base generator's subgroup. To ensure that points remain in the generator subgroup, an extra step is required to *clear the cofactor* by multiplying the resulting point by the cofactor (Curve25519's cofactor is 8).

The final formula for generating the j^{th} chunk's generator is:

$$G_j = 8 \cdot \text{Map}(h(\text{timestamp}) + j), \quad \text{for } j = 1, \dots, 7 \quad (1)$$

4.2.2 Client. To send a message to a specific destination, the client first randomly chooses a path through the mixnet and generates a nonce. Using this nonce and the public keys of each mixnode along the path, the client derives a chain of shared secret integers (s_1, s_2, s_3) as follows:

$$(s_1, s_2, s_3) \left\{ \begin{array}{l} \alpha_i = x_i G \\ S_i = x_i K_i \\ s_i = \text{Map}^{-1}(S_i) \\ r_i = h(\alpha_{i,y} \parallel S_{i,y}) \\ x_{i+1} = x_i r_i \pmod{n} \end{array} \right. \quad (2)$$

where x_1 is the client's nonce, K_i is the public key of the i^{th} mixnode in the path ($K_i = k_i G$), n is the subgroup order (i.e. $(n+1)G = G$), $\parallel$ is the concatenation operator and subscript y refers to the y -coordinate of the point. The scalar r_i acts as a pseudo-randomizer to update the secret x_i for the next hop, ensuring that each cryptographic element α_i remains unlinkable across mixnodes.

The IP addresses of the mixnodes and the destination are padded with random bits then mapped to elliptic curve points using Elligator, allowing later recovery of these addresses. Both the resulting IP points and the shared secrets are split into shares and distributed to m different TTPs. In addition, the client send the hash of each shared secret to these TTPs.

Each TTP then independently computes a partial header from the received shares and returns it to the client. The client aggregates these partial headers by performing chunk-wise point additions to reconstruct the complete encrypted header. Finally, the client sends the header along with α_1 to the first mixnode from the path.

4.2.3 TTP. They receive from the client:

- a share of the destination address (EC point noted Δ);

- a share of each mixnode address in the path (EC points noted N_i);
- a share of each shared secrets (integers noted s_i);
- the hash of each shared secrets (integers noted σ_i).

The TTP constructs a partial encrypted header by layering routing information in reverse path order, starting from the last mixnode, as illustrated by Figure 3.

At each layer, two new chunks are introduced: the mixnode's address (N_i) and the integrity tag (γ_i). To preserve a fixed-length header while allowing mixnodes to reverse the operation (as in the original design), four additional *filler chunks* are appended. These filler chunks are carefully computed such that at each layer, after applying the corresponding masking points ($s_i G_j$), the last two chunks cancel out and become identity point (i.e. point at infinity). This mechanism allows these two chunks to be safely truncated at each layer, ensuring a consistent header size while preserving reversibility for mixnodes.

To initialize the header, the TTP encrypts the destination point using the shared secret of the last mixnode and append the required filler chunks. Let Δ be the partial destination point, s_i the partial shared secret corresponding to the i^{th} mixnode in the path, and G_j the j^{th} chunk's generator. The chunks for the last layer (assuming a 3-hop path) are computed as follows:

$$\beta_3 \begin{cases} \beta_{31} = \Delta + s_3 G_1 \\ \beta_{32} = -(s_2 G_4 + s_1 G_6) \\ \beta_{33} = -(s_2 G_5 + s_1 G_7) \\ \beta_{34} = -s_2 G_6 \\ \beta_{35} = -s_2 G_7 \end{cases} \quad (3)$$

Subsequent layers ($i = 2$ and $i = 1$) are computed with Eq. 4 where N_{i+1} represents the elliptic curve point corresponding to the $(i+1)^{\text{th}}$ mixnode in the path and γ_{i+1} is the integrity tag from previous layer.

$$\beta_i \begin{cases} \beta_{i1} = N_{i+1} + s_i G_1 \\ \beta_{i2} = \gamma_{i+1} + s_i G_2 \\ \beta_{i3} = \beta_{(i+1,1)} + s_i G_3 \\ \beta_{i4} = \beta_{(i+1,2)} + s_i G_4 \\ \beta_{i5} = \beta_{(i+1,3)} + s_i G_5 \end{cases} \quad (4)$$

For each layer, the integrity tag is computed with Eq. 5 where $h^{(j)}(\sigma_i)$ means hashing j times σ_i which is the hash of the i^{th} mixnode's complete shared secret. The integrity tag consists in a pseudo-random weighted sum of the chunks, offset by a secret point derived from the shared secret integer. The pseudo-random weights (i.e. $h^{(j)}(\sigma_i)$) ensure binding, preventing adversaries from modifying individual chunks. Without these weights, an attacker could add a random point P to one chunk and subtract P from another, thereby preserving the overall sum and bypassing the integrity check. The secret offset (i.e. $s_i G$) prevents curious TTPs from correlating integrity tags. Since TTPs are aware of these pseudo-random weights, omitting this would allow them to potentially link some incoming and outgoing mixnode's packets by simply verifying

integrity tags.

$$\gamma_i = s_i G + \sum_{j=1}^5 (h^{(j)}(\sigma_i) \beta_{ij}) \quad (5)$$

When the first layer is finally computed, it is appended with the integrity tag γ_1 and the first mixnode's point N_1 . Finally, the TTP returns the constructed partial header to the client.

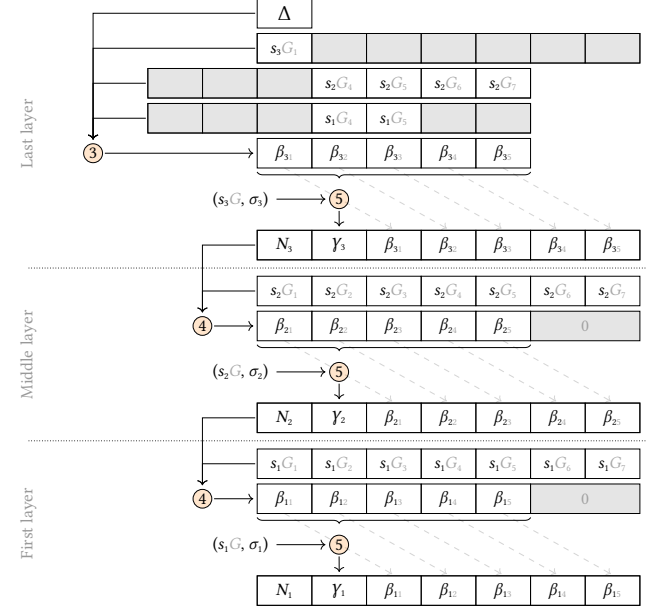


Figure 3: Chunk-wise header encryption (TTP side), where $\otimes$ refers to Eq. x.

Security Note. To preserve unlinkability, it is essential that the generators ($G_1, \dots, G_5$) are *independent*, meaning that their scalar relationships are unknown and cannot be derived. If the same generator G were used across chunks (or their scalar relationships known), an adversary could compute the chunk-wise difference between consecutive layers: $\beta_{ij} - \beta_{i+1,j} = s_i G$. Since the shared secret s_i remains consistent within a layer, the resulting differences would reveal a predictable pattern (uniform or preserved scalar relationships). This consistency could allow an adversary to correlate incoming and outgoing packets at a mixnode, thereby breaking the unlinkability property. Using independent generators for each chunk prevents such correlations and is therefore critical to the protocol's security.

4.2.4 Mixnode. When a packet arrives at mixnode i , it begins by extracting the relevant header fields, namely the cryptographic element α_i and the integrity tag γ_i . The mixnode then recomputes the shared secret point S_i using its private key k_i and the cryptographic element α_i :

$$\begin{aligned} S_i &= k_i \alpha_i \\ s_i &= \text{Map}^{-1}(S_i) \end{aligned} \quad (6)$$

This shared secret point is subsequently mapped to the shared integer s_i using Elligator inverse mapping. This shared integer is

then used to verify the integrity tag (Eq. 7).

$$Y_i \stackrel{?}{=} s_i G + \sum_{j=1}^5 (h^{(j)}(s_i) \beta_{ij}) \quad (7)$$

If the integrity check passes, the mixnode pads the header by appending two *identity* chunks (i.e. point at infinity) and then updates it as:

$$\beta'_{ij} = \beta_{ij} - s_i G_j \quad \forall j = 1, \dots, 7 \quad (8)$$

The first chunk (β'_{i1}), encodes the next mixnode's IP address (N_{i+1}). This point is mapped back to an integer using Elligator and the random padding is removed by keeping the last 128 bits (i.e. size of an IP address).

The second chunk (β'_{i2}) is the next integrity tag (Y_{i+1}) for the remaining five chunks that form the new encrypted routing information (β_{i+1}).

To maintain unlinkability, the cryptographic element α_i must be updated for the next hop. As in the client's step, it is computed using the y -coordinates of both α_i and S_i :

$$\alpha_{i+1} = \text{hash}(\alpha_{iy} \parallel S_{iy}) \alpha_i \quad (9)$$

Finally, the mixnode forwards the updated header ($\alpha_{i+1}, Y_{i+1}, \beta_{i+1}$) to the next node.

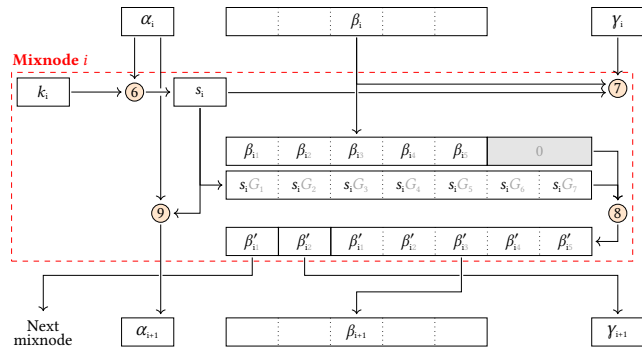


Figure 4: Processing sphinx header at mixnode i , where $\otimes$ refers to Eq. x.

5 Evaluation

Our prototype implementation is available on GitHub¹. The implementation is written in Python 3.13 using the ECPy library for elliptic curve operations. ECPy is well-suited for prototyping due to its simplicity, but it is significantly slower than many EC python libraries (approximately one order of magnitude slower). Nevertheless, it remains one of the few Python libraries supporting Montgomery Curve25519, which is essential for our implementation.

We evaluate our system on two main axes: computational complexity and unlinkability (i.e. resistance to correlation between incoming and outgoing packets).

¹<https://github.com/AurelienCha/Decentralized-Sphinx>

Table 1: Number of expensive cryptographic operations per party, where p is the path length (e.g. $p = 3$), and m is the number of TTPs.

	Client	TTP	Mixnode
EC Multiplication	$O(p m)$	$O(p^2)$	$O(p)$
EC Addition	$O(p m)$	$O(p^2)$	$O(p)$
Point $\rightarrow$ Integer	$O(p)$	0	2
Integer $\rightarrow$ Point	$O(p)$	0	0
Hash	$O(p)$	$O(p^2)$	$O(p)$

5.1 Computational Complexity

Instead of measuring raw execution time, which can vary significantly based on hardware and software environments, we focus on the number of expensive cryptographic operations. Table 1 counts these operations per party for m TTPs and path of length p . To compute the total cost for sending one message, the TTP cost should be multiplied by m and the mixnode cost by p .

Since long paths are typically unnecessary, we can approximate $O(p)$ as constant (i.e. $O(1)$). A path of length $p = 3$ is generally sufficient to ensure strong anonymity.

Aurelien: Iness do you have some source to support this ? If yes, could you cite here, thanks.

Consequently, the overall computational burden is more sensitive to the number of TTPs m rather than the path length. Since our design remains secure as long as a single TTP is honest (i.e. does not collude), only a small number of TTPs may be required. A deeper analysis of how many TTPs are needed to ensure a desired level of trust remains an open question for future work.

5.2 Unlinkability Evaluation

Quantifying unlinkability is inherently challenging. However, under the assumption that uniformly random packet headers prevent adversaries from correlating incoming and outgoing packets through cryptanalysis, unlinkability can be approximated by assessing the statistical randomness of the headers. For this purpose, we rely on the NIST SP 800-22 statistical test suite [15], which consists of 15 tests designed to evaluate the quality of random number generators. Each test produces a p-value representing how likely the data could be produced by a uniform source. If the data are truly random, the distribution of p-values across many samples should itself follow a uniform distribution.

We compare the statistical results of our implementation with those of the original Sphinx implementation by Danezis². All graphs presented in this section correspond to the results of the NIST statistical test suite applied to 20 distinct datasets. Each dataset is generated under a different setup configuration to assess the variability and robustness of our implementation.

To generate a dataset, we simulate a network of 20 mixnodes and 3 trusted third parties (TTPs). For each iteration, a random destination IP address and a path consisting of 3 mixnodes are selected. The client then generates shares and distributes them to the three TTPs, which independently compute partial headers.

²<https://github.com/UCL-InfoSec/sphinx>

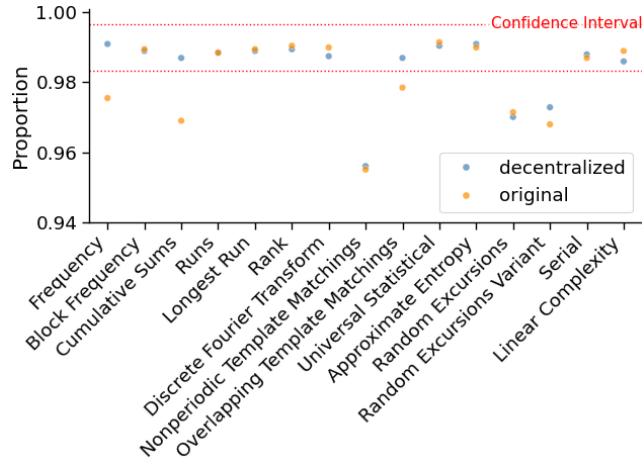


Figure 5: Proportion of successful sequences from our implementation and the original Sphinx implementation over the fifteen NIST SP 800-22 statistical tests (significance level = 0.01).

These headers are then aggregated and forwarded through the simulated mixnet.

According to the NIST documentation, the statistical test suite requires input sequences of 1 000 000 bits. To meet this requirement, generated headers are concatenated until the total length reaches 1 000 000 bits. Each dataset contains 100 of those concatenated sequences.

NIST outlines two approaches for interpreting the empirical results of its statistical test suite. The first assesses the proportion of sequences that pass each statistical test, as shown in Fig.5. The second evaluates whether the distribution of p-values is statistically uniform, as would be expected from random sequences. These results are presented in Fig.7.

Figure 5 shows the proportion of sequences that successfully pass each statistical test for both our implementation and the original Sphinx implementation. Following the NIST guidelines, the significance level is set to $\alpha = 0.01$, and the confidence interval is defined as:

$$(1 - \alpha) \pm 3\sqrt{\frac{\alpha(1 - \alpha)}{m}},$$

where m denotes the number of samples.

Our implementation fails only three tests: *Nonperiodic Template Matching*, *Random Excursions*, and *Random Excursions Variant*, all of which are also failed by the original Sphinx implementation. In contrast, our implementation successfully passes three tests that the original implementation fails: *Frequency*, *Cumulative Sums*, and *Overlapping Template Matching*.

Figure 6 presents the distribution of p-values obtained for each test. This figure is intended solely for visual inspection. A Chi-square test is performed on each histogram to assess their uniformity. The results are shown in Figure 7, with a significance threshold of 0.0001 as recommended by the NIST guidelines.

Table 2: Statistical test summary for both implementations (● = Pass, ○ = Fail)

Statistical Test	Decentralized		Original	
	Excessive Rejections	Lacks Uniformity	Excessive Rejections	Lacks Uniformity
Frequency	●	●	○	○
Block Frequency	●	●	●	●
Cumulative Sums	●	●	○	○
Runs	●	●	●	●
Longest Run	●	●	●	●
Rank	●	●	●	●
Discrete Fourier Transform	●	●	●	●
Nonperiodic Template Match.	○	●	○	●
Overlapping Template Match.	●	●	○	●
Universal Statistical	●	●	●	●
Approximate Entropy	●	●	●	●
Random Excursions	○	●	○	●
Random Excursions Variant	○	●	○	●
Serial	●	●	●	●
Linear Complexity	●	●	●	●

Two Chi-square results stand out: the *Frequency* and *Cumulative Sums* tests. Both were successful for our implementation, despite failing for the original Sphinx implementation.

5.3 Discussion

In terms of computational complexity, our implementation is slower than the original Sphinx implementation. However, in the context of a mixnet where inherent latency is already expected, this additional delay is likely acceptable.

Regarding unlinkability (i.e. output randomness), our implementation performs slightly better than the original implementation, as it passes more statistical tests, as shown in Table 2. Since the original Sphinx implementation is widely considered as secure, and assuming no specific vulnerabilities exist in our design, we can reasonably infer that our approach maintains a comparable level of security.

Come back to 3.3 objectives and explain why they hold. If we have research questions from 2, answer them. Summarize overheads impacts and discuss feasibility in the context of Nym.

Aurelien: It is quite difficult to refer to these objectives because the paper in this state does "half" of the work. We only decentralized the construction of the Sphinx header. After that, we still need to provide the actual "credentials". It's just way easier to do it now since we can "verify/link" the credential with "partial destination" (with the original implementation we would have to somehow verify/link this credential to a destination that have been encrypted under multiple layer.)

We have proposed a decentralized scheme that delegates the construction of the Sphinx header to a set of Trusted Third Parties (TTPs), ensuring that no single TTP can infer meaningful information from the computation, even in the case of collusion (if at least one party remains honest). Since these TTPs directly access partial destination IP information, it becomes significantly easier to issue anonymous credentials for specific usages, while preventing users from cheating on their intended destinations. The detailed construction of such credentials, lies beyond the scope of this paper.

Another future work is to evaluate the computational overhead introduced by the TTPs in comparison to the computational waste

move to Con work

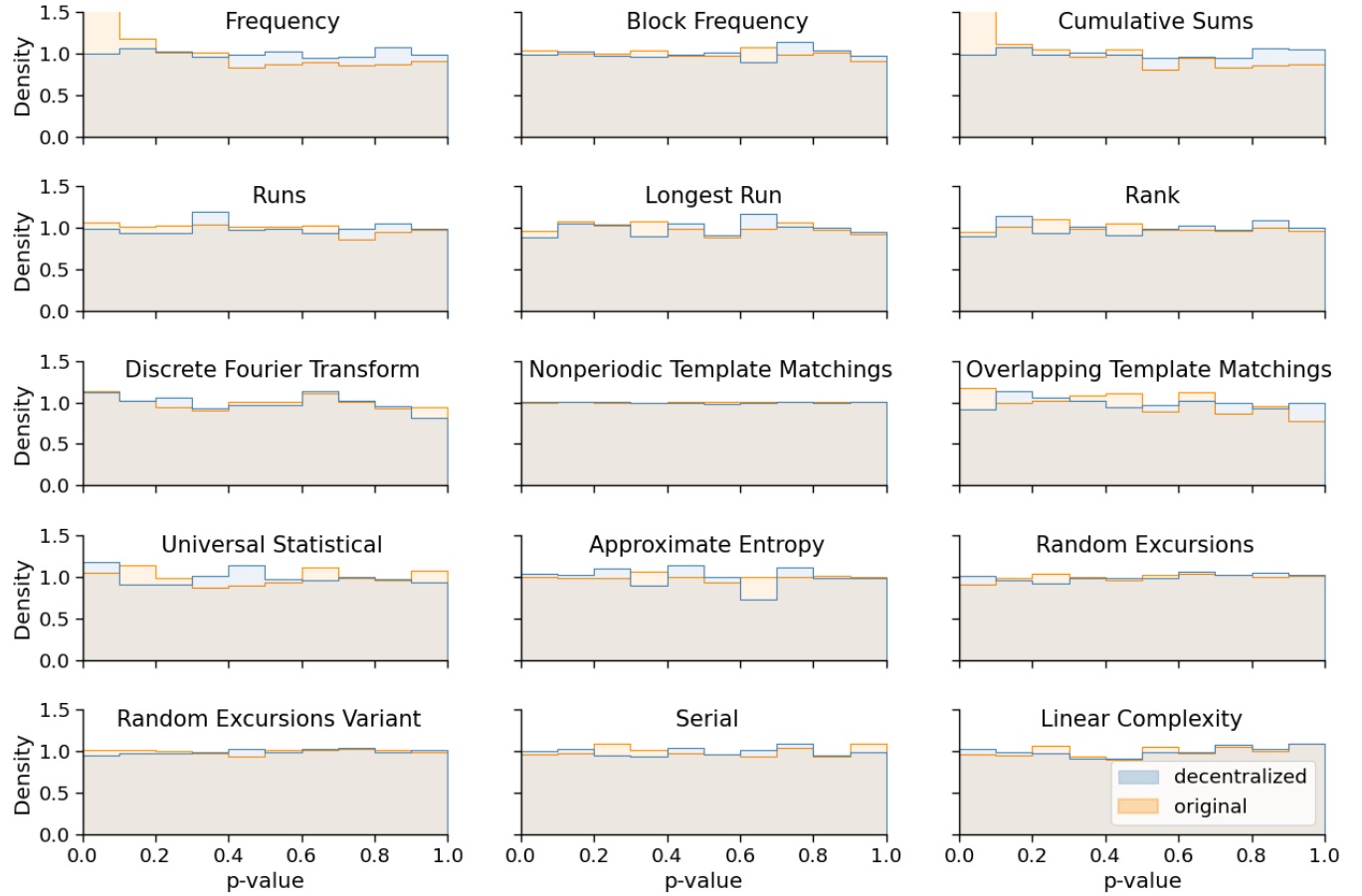


Figure 6: P-value distribution comparison between our implementation and the original Sphinx across the fifteen NIST SP 800-22 statistical tests.

caused by malicious users in the original Sphinx implementation, in order to assess the overall trade-off.

From our call 2025-06-10: “While it’s computationally more complex, our schema does not allow the mixnet to be saturated by an attacker.”

6 Conclusion

Acknowledgments

This research is funded by Belgium Walloon region CyberExcellence program (grant #2110186)

References

- [1] Nikolaos Alexopoulos, Aggelos Kiayias, Riivo Talviste, and Thomas Zacharias. 2017. MCMix: Anonymous Messaging via Secure Multiparty Computation. In *26th {USENIX} Security Symposium ({USENIX} Security 17)*. 1217–1234. <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/alexopoulos>
- [2] Iness BEN GUIRAT, Debajyoti Das, and Claudia Diaz. 2023. Blending different latency traffic with beta mixing. *Proceedings on Privacy Enhancing Technologies* (2023).
- [3] David Chaum. 1981. Untraceable Electronic Mail, Return Addresses, and Digital Pseudonyms. *Commun. ACM* 24, 2 (1981), 84–88. doi:10.1145/358549.358563
- [4] David Chaum, Farid Javani, Aniket Kate, Anna Krasnova, Joeri Ruiter, Alan T Sherman, and Debajyoti Das. 2016. *cMix: Anonymization by high-performance scalable mixing*. Technical Report. Technical report. <http://www.cs.bham.ac.uk/~deruitej/papers/cmixon.pdf>
- [5] Lance Cottrell. 1995. Mixmaster and remailer attacks. <https://www.obscure.com/~loki/remailer-essay.html>
- [6] George Danezis, Roger Dingledine, and Nick Mathewson. 2003. Mixminion: Design of a Type III Anonymous Remailer Protocol. In *Proceedings of the 2003 IEEE Symposium on Security and Privacy*. IEEE, 2–15.
- [7] George Danezis and Ian Goldberg. 2009. Sphinx: A Compact and Provably Secure Mix Format. In *2009 30th IEEE Symposium on Security and Privacy*. 269–282. <http://research.microsoft.com/en-us/um/people/gdane/papers/sphinx-eprint.pdf>
- [8] Claudia Diaz, Harry Halpin, and Aggelos Kiayias. 2021. The Nym Network. <https://nymtech.net/nym-whitepaper.pdf>, 38 pages.
- [9] Roger Dingledine and Nick Mathewson. 2006. *Anonymousity Loves Company: Usability and the Network Effect.. In WEIS*.
- [10] David M. Goldschlag, Michael G. Reed, and Paul F. Syverson. 1996. Hiding Routing Information. In *Proceedings of the First International Workshop on Information Hiding*. Springer-Verlag, Berlin, Heidelberg, 137–150.
- [11] Jelle Van Den Hooff, David Lazar, Matei Zaharia, and Nickolai Zeldovich. 2015. Vuvuzela: Scalable private messaging resistant to traffic analysis. In *Proceedings of the 25th Symposium on Operating Systems Principles*. 137–152. <https://people.csail.mit.edu/nickolai/papers/vandenhooff-vuvuzela.pdf>
- [12] Albert Kwon, David Lu, and Srinivas Devadas. 2020. {XRD}: Scalable messaging system with cryptographic privacy. In *17th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 20)*. 759–776.
- [13] David Lazar, Yossi Gilad, and Nickolai Zeldovich. 2018. Karaoke: Distributed private messaging immune to passive traffic analysis. In *13th {USENIX} Symposium on Operating Systems Design and Implementation ({OSDI} 18)*. 711–725.

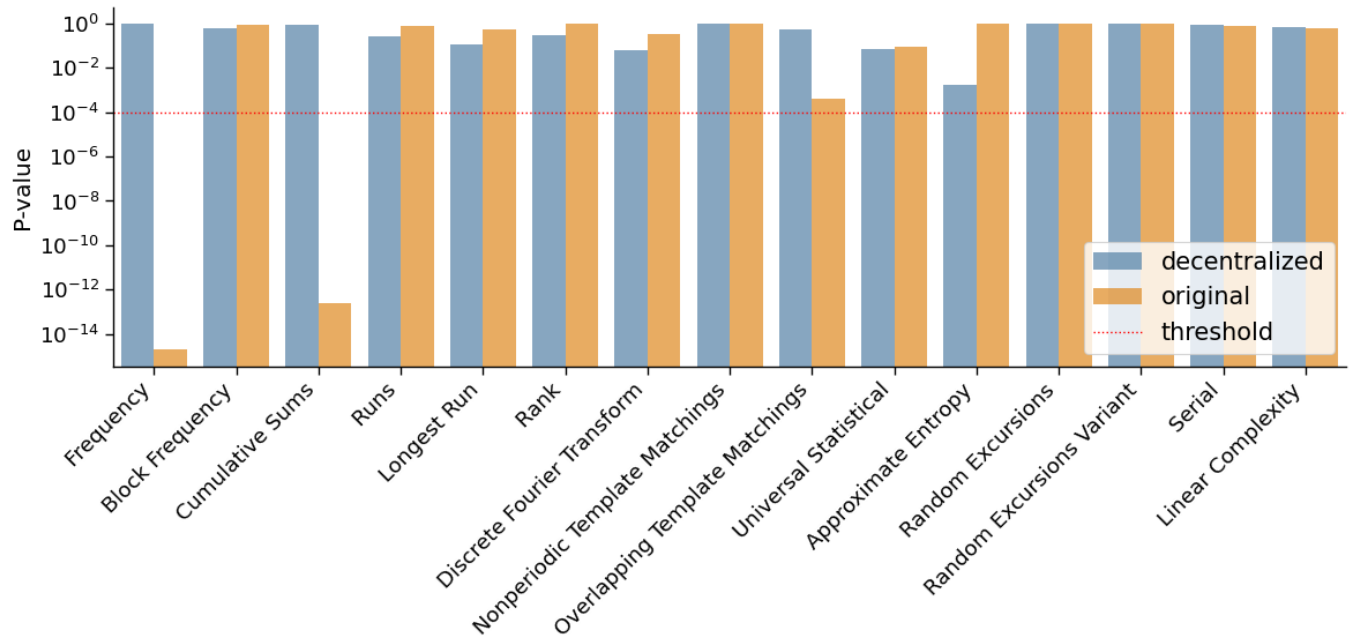


Figure 7: Chi-square test assessing the uniformity of p-value distributions for each dataset across the fifteen NIST SP 800-22 statistical tests.

- [14] Ulf Möller, Lance Cottrell, Peter Palfrader, and Len Sassaman. 2003. Mixmaster Protocol — Version 2. IETF Internet Draft.
- [15] NIST. 2010. *A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications*. Special Publication SP 800-22. National Institute of Standards and Technology. doi:10.6028/NIST.SP.800-22r1a
- [16] Sameer Parekh. 1996. Prospects for remailers. *First Monday* 1 (August 1996). Issue 2.

- [17] Ania M Piotrowska, Jamie Hayes, Tariq Elahi, Sebastian Meiser, and George Danezis. 2017. The loopix anonymity system. In *26th {USENIX} Security Symposium ({USENIX} Security 17)*. 1199–1216.
- [18] Alberto Sonnino, Mustafa Al-Bassam, Shehar Bano, Sarah Meiklejohn, and George Danezis. 2019. Coconut: Threshold issuance selective disclosure credentials with applications to distributed ledgers. In *26th Annual Network and Distributed System Security Symposium, NDSS 2019*. The Internet Society.

Received 27 June 2025; revised XX July 2025; accepted XX July 2025